Feature extraction

Giữ encoder cố định, dùng lại vector

Tìm lại một ảnh hoặc tài liệu

Ảnh lệch vài pixel vẫn cùng nội dung.

“Shortest path” và “đường ngắn nhất” gần về chủ đề.

Có thể để một bộ đọc cố định tạo vector, rồi dùng kNN hoặc head nhỏ không?

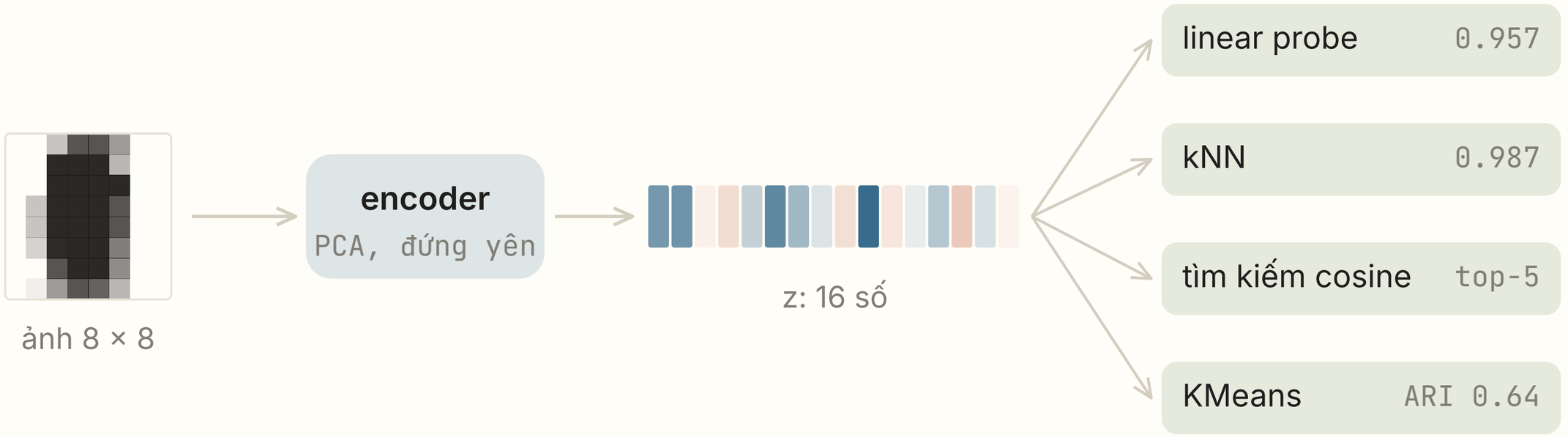
Mục tiêu

- Lấy embedding từ model pretrained.
- Dùng embedding với logistic regression, kNN và tìm ảnh giống nhau.
- Tính cosine similarity và pooling có mask.
- Tránh leakage khi chọn encoder và xử lý dữ liệu.

01

Biểu diễn cố định

Trích một lần, dùng nhiều lần



Một encoder đứng yên cho embedding; dùng lại cho phân loại, kNN, tìm kiếm, phân cụm.

Encoder là một hàm

$$\mathbf{z} = f_\theta(\mathbf{x}) \in \mathbb{R}^d$$

Feature extraction giữ θ cố định.

Embedding không tự bảo đảm gần về nghĩa: điều đó phụ thuộc mục tiêu học.

HỎI NHANH

Encoder cho mỗi ảnh một vector 512 số, và có 100 ảnh. Ma trận embedding xếp mỗi mẫu một hàng có shape gì?

A `(512, 100)`

B `(100, 512)`

C `(100,)`

D `(1, 51200)`

Lấy vector ở đâu?

Ảnh → backbone → **vector trước head** → head → logits nhãn cũ.

Logits trả lời câu hỏi của nhãn cũ; đặc trưng thường lấy trước head.

Nhiều token: cần quy tắc pooling, ghi lại như một phần thí nghiệm.

HỎI NHANH

ResNet-50 học trên ImageNet có head trả về 1000 logits, và ngay trước head là một vector 2048 số. Câu lạc bộ cần phân loại ảnh thành 5 loại linh kiện máy tính, không loại nào trùng tên lớp ImageNet. Nên lấy đặc trưng nào cho linear probe?

A 1000 logits, vì đó là tầng có ý nghĩa rõ nhất

B Xác suất softmax của 1000 lớp, vì đã chuẩn hóa tổng bằng 1

C Vector 2048 số trước head, rồi học một head 5 lớp mới

D Pixel thô, vì nhãn mới không có trong ImageNet

Encoder PCA học tại chỗ

Ảnh chữ số 8×8 có 64 pixel; PCA còn 16 tọa độ.

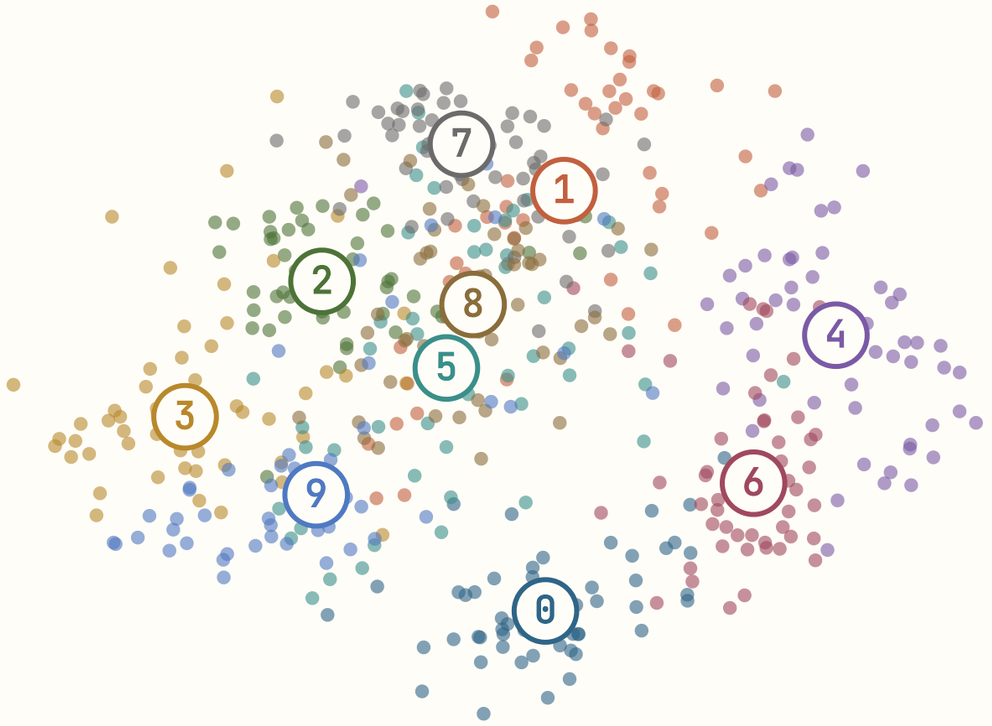
PCA học một lần trên train rồi đứng yên: vai trò của encoder pretrained, chỉ là học tại chỗ.

```
encoder = PCA(
    n_components=16, svd_solver="full")
Z_train = encoder.fit_transform(X_train)
Z_valid = encoder.transform(X_valid)
print("embedding:",
      Z_train.shape, Z_valid.shape)
```

KẾT QUẢ

```
embedding: (1257, 16) (540, 16)
```

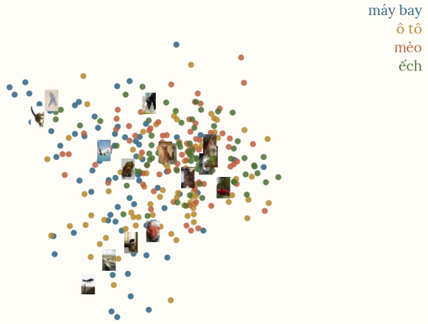
Embedding của 540 ảnh validation



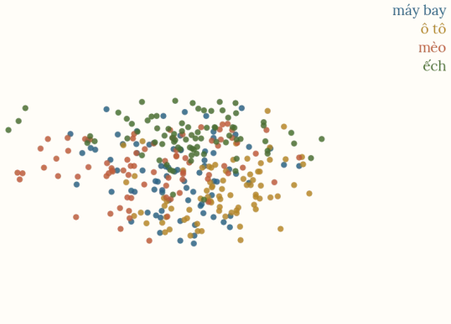
hai tọa độ đầu
trong 16 của z

Hai tọa độ đầu của embedding: cùng chữ số tụ lại gần nhau.

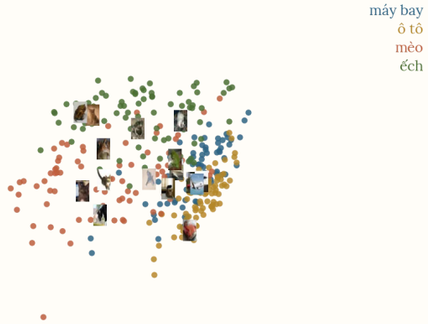
Từ pixel sang đặc trưng



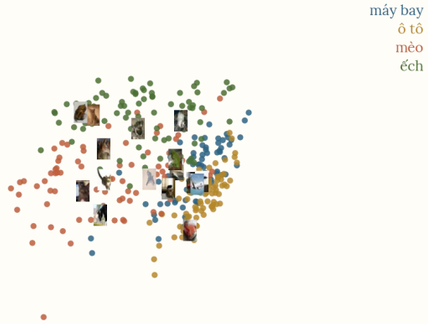
PCA của pixel thô (3072 số mỗi ảnh): bốn lớp trộn vào nhau



chuyển từ không gian pixel sang không gian đặc trưng



PCA của vector ResNet-18 (512 số mỗi ảnh): cùng lớp tụ lại



linear probe trên vector ResNet-18: accuracy 5-fold 0.90

Ảnh CIFAR-10 theo PCA của pixel thô, rồi theo PCA của vector đặc trưng ResNet-18.

Chia trước, fit sau

PCA không dùng nhãn nhưng vẫn học từ input: chỉ fit trên train.

ID không trùng chưa đủ: ảnh gần trùng (cùng buổi, cùng nguồn) phải nằm cùng một phía.

HỎI NHANH

Kho ảnh câu lạc bộ có 40 buổi hoạt động, mỗi buổi khoảng 50 ảnh chụp liên tiếp. Chia ngẫu nhiên theo từng ảnh thì kNN trên embedding đạt 0.99 trên validation, nhưng dùng cho một buổi mới thì kém hẳn. Nguyên nhân hợp lý nhất là gì?

A Encoder đã quá khớp với ảnh của câu lạc bộ

B Những ảnh gần như trùng nhau của cùng một buổi nằm ở cả hai tập

C kNN cần k lớn hơn khi kho có hơn 1000 ảnh

D Embedding chưa được chuẩn hóa L2 trước khi tìm láng giềng

02

Học hoặc tìm trên vector

Linear probe

$$\boldsymbol{\ell} = W\mathbf{z} + \mathbf{b}$$

Giữ encoder, học head từ nhãn mới.

16 đầu vào, 10 lớp có bias: **170 tham số head**.

```
probe = LogisticRegression(C=1.0, max_iter=1000)
probe.fit(Z_train, y_train)
```

Đây là head tuyến tính; không phải fine-tuning encoder.

HỎI NHANH

Trong linear probe, những tham số nào được học bằng nhãn của bài mới?

A Chỉ W và $\mathbf{b}$ của head, encoder giữ nguyên

B Cả encoder lẫn head, encoder với learning rate nhỏ

C Chỉ encoder, còn head là logistic regression cố định

D Không tham số nào, vì probe chỉ đo khoảng cách

Cùng validation, ba cách dùng

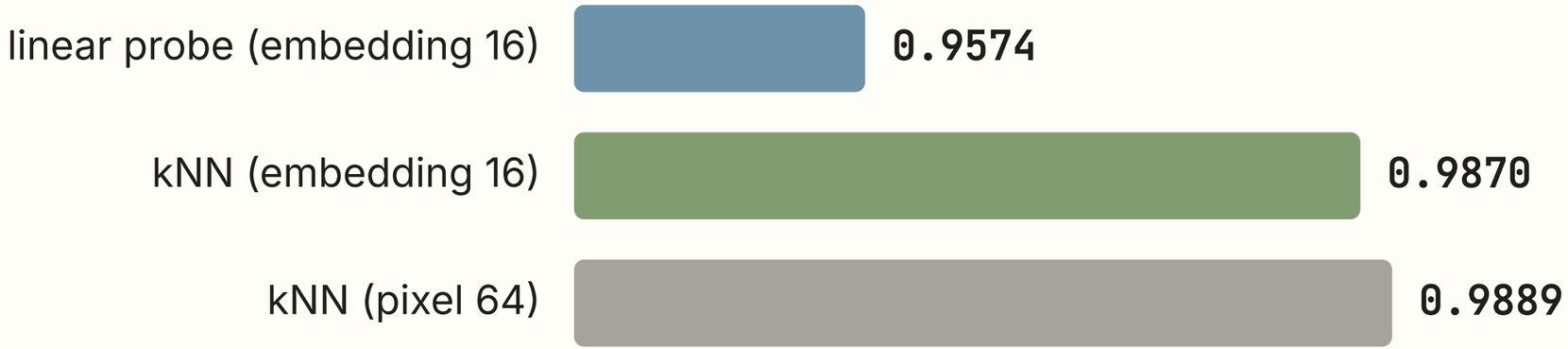
Cách	Accuracy
Linear probe PCA	0.9574
kNN embedding	0.9870
kNN pixel	0.9889

540 ảnh validation, cùng split.

kNN: 16 tọa độ gần bằng 64 pixel, chênh đúng 1 ảnh.

Probe thấp hơn: lớp gom cụm tốt nhưng khó tách bằng siêu phẳng.

Ba accuracy



trục bắt đầu từ 0.94; 0.0019 ≈ đúng một ảnh trên 540

kNN trên 16 tọa độ PCA gần bằng kNN trên đủ 64 pixel.

HỎI NHANH

Một bạn đọc bảng trên và kết luận: “kNN trên pixel đạt 0.9889, cao hơn kNN trên embedding là 0.9870, vậy trích đặc trưng làm hại, nên bỏ encoder”. Phản biện nào đúng nhất?

A Kết luận đúng, vì cùng validation thì số lớn hơn là cách tốt hơn

B Kết luận sai, vì embedding luôn tốt hơn pixel khi có encoder

C Hai cách chỉ chênh đúng một ảnh trên 540, một cách chia chưa đủ để kết luận

D Kết luận sai, vì phải so linear probe với kNN trên pixel mới công bằng

Cosine của (3, 4) và (6, 8)

$$\mathbf{a} = (3, 4), \quad \mathbf{b} = (6, 8)$$

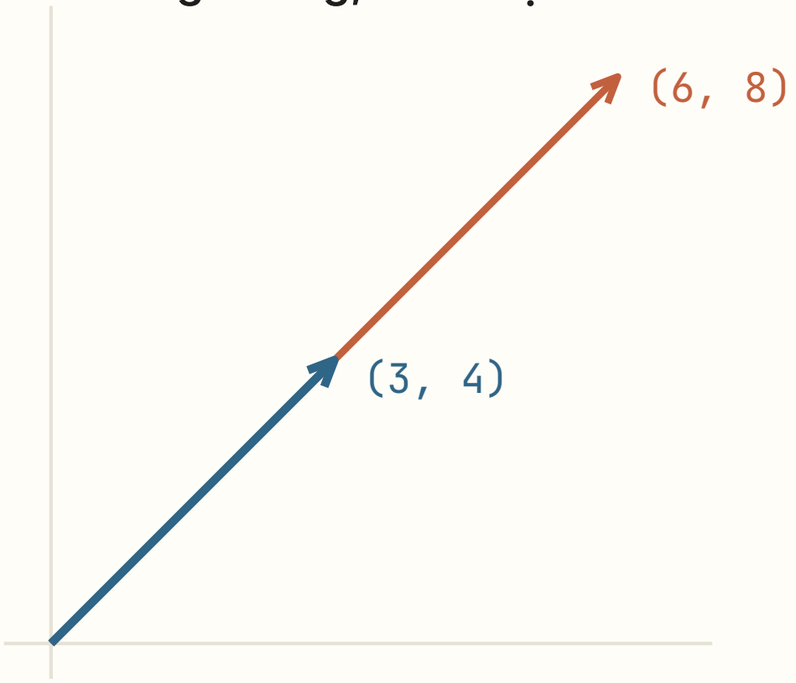
$$\mathbf{a}^\top \mathbf{b} = 50, \quad \|\mathbf{a}\| = 5, \quad \|\mathbf{b}\| = 10$$

$$\cos(\mathbf{a}, \mathbf{b}) = 1, \qquad \|\mathbf{a} - \mathbf{b}\| = 5$$

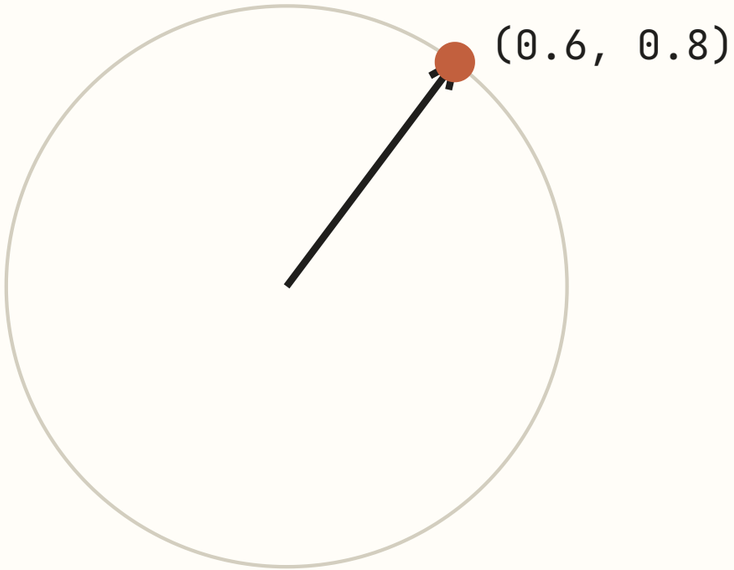
Khi mọi vector dài 1: $\|\mathbf{a} - \mathbf{b}\|^2 = 2 - 2\cos.$

Cosine bỏ ảnh hưởng độ dài

cùng hướng, khác độ dài



sau chuẩn hóa: trùng nhau



`cos = 1`

`Euclid = 5`

(3, 4) và (6, 8) cùng hướng nên cosine bằng 1, dù khoảng cách Euclid là 5.

HỎI NHANH

Mọi vector đã được chuẩn hóa về độ dài 1. Hàng A có cosine 0.9 với truy vấn, hàng B có cosine 0. Bình phương khoảng cách Euclid từ truy vấn tới A và tới B lần lượt là bao nhiêu?

A 0.1 và 1.0

B 1.8 và 0

C 0.9 và 0

D 0.2 và 2.0

Tìm trong kho train

Mọi vector đã chuẩn hóa độ dài 1, nên tích vô hướng là cosine.

```
similarity = (query @ database.T)[0]
nearest = np.argsort(
    -similarity, kind="stable")[:5]
print("nhan 5 hang gan:",
      y_train[nearest].tolist())
```

KẾT QUẢ

```
nhan 5 hang gan: [0, 0, 0, 0, 0]
```

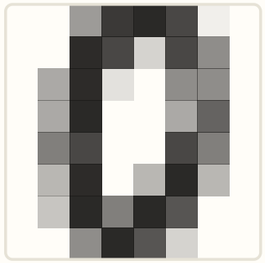
Truy vấn là một chữ số 0; năm hàng gần nhất cũng là 0.

Cosine: 0.972, 0.934, 0.931, 0.915, 0.905.

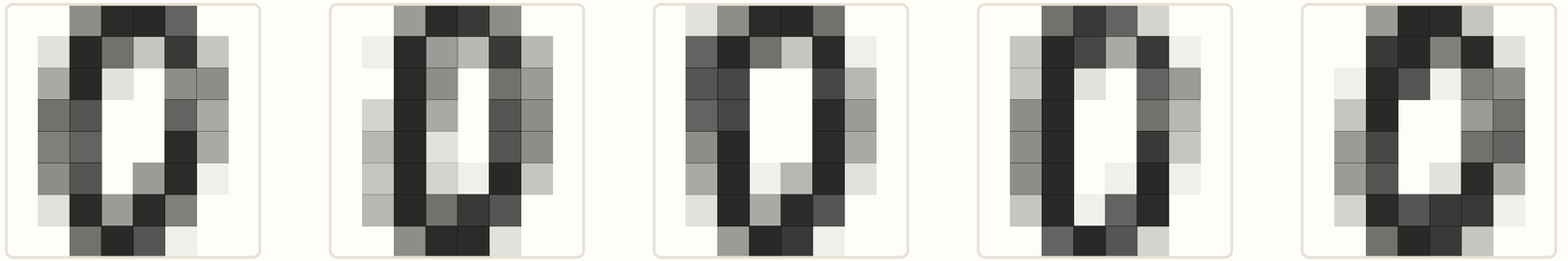
Một truy vấn chưa đo được cả hệ tìm kiếm.

Tìm ảnh gần nhau trong embedding

truy vấn



5 ảnh train gần nhất theo cosine



0.972 0.934 0.931 0.915 0.905

đều là chữ số 0

Năm ảnh train gần nhất với một chữ số 0 theo cosine: cả năm đều là 0.

HỎI NHANH

`similarity = np.array([0.2, 0.9, -0.4, 0.7])` là cosine giữa truy vấn và bốn hàng của kho. `np.argsort(-similarity)[:2]` trả về gì?

A `[1, 3]`

B `[2, 0]`

C `[0.9, 0.7]`

D `[3, 1]`

03

Pooling, cụm và cache

Từ token đến câu

`hidden`: `(B,L,D)`; sau pooling còn `(B,D)`.

$$\mathbf{z} = \frac{\sum_t m_t \mathbf{h}_t}{\sum_t m_t}$$

Mask chọn token thật; padding không được góp vào tổng.

Một padding phá trung bình

Token thật: `(1,3)`, `(3,1)`.

Padding giả lập: `(999,999)`.

KẾT QUẢ

```
masked mean: [[2.    2.   ]
 [1.333 1.333]]
mean sai: [[334.333 334.333]
 [  1.333   1.333]]
doi chieu: True
```

Lọc token bằng vòng lặp khớp với bản tensor có mask.

HỎI NHANH

Mask của một câu là `(1, 1, 1, 0, 0)`, và tổng ba vector của các token thật là `(6, 3)`. Masked mean bằng bao nhiêu?

A `(1.2, 0.6)`

B `(6, 3)`

C Còn tùy giá trị của hai vector PAD

D `(2, 1)`

HỎI NHANH

`hidden` có shape `(32, 20, 768)`, `mask` có shape `(32, 20)`. Dòng `(hidden * mask).sum(dim=1)` làm gì?

A Chạy đúng, cho tổng các token thật với shape `(32, 768)`

B Báo lỗi shape, phải dùng `mask.unsqueeze(-1)`

C Chạy được, nhưng cộng cả padding vào tổng

D Cho shape `(32, 20)`, vì nhân theo từng token

Clustering không có tên lớp sẵn

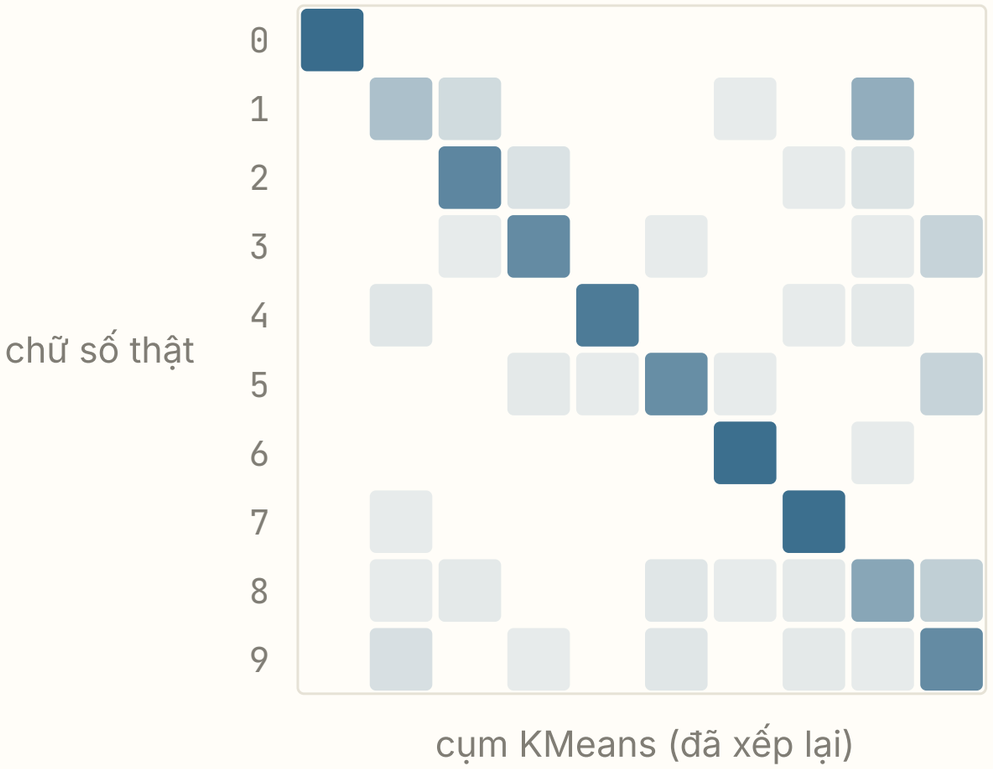
```
clusterer = KMeans(
    n_clusters=10, n_init=10, random_state=0)
clusterer.fit(Z_train)
valid_clusters = clusterer.predict(Z_valid)
```

ARI validation: **0.6364**.

So thẳng số cụm với nhãn: accuracy **0.1074**.

KMeans đánh số cụm tùy ý; ARI không đổi khi hoán vị tên cụm.

Chữ số × cụm



ARI 0.636

số cụm ≠ chữ số:
so thẳng chỉ 0.107

Chữ số thật × cụm KMeans: phần lớn nằm trên đường chéo, vài chữ số bị tách hoặc gộp.

HỎI NHANH

Muốn biết mỗi cụm ứng với chữ số nào để tính một accuracy dễ đọc cho phép phân cụm ở trên, ta làm thế nào cho đúng?

A Gán cho mỗi cụm nhãn chiếm đa số trên train, rồi áp lên validation

B Gán cho mỗi cụm nhãn chiếm đa số trên chính validation rồi chấm

C Chạy lại KMeans với `random_state` khác tới khi accuracy cao

D Không làm được, vì KMeans không nhận nhãn khi học

Cache không chỉ là một mảng số

- ID và split.
- Model/revision, processor, pooling.
- Quy tắc normalization.
- Encoder thay đổi thì phải tính lại embedding.

Encoder có BatchNorm muốn cố định còn cần `eval()`.

Khi đi thi

- Trích một lần, lưu kèm ID; baseline probe hoặc kNN.
- PCA, scaler, head fit trong từng fold.
- Đề cấm pretrained: tự học encoder.

04

Luyện đề

LUYỆN ĐỀ 1

Linear probe hai lớp là logistic regression trên $\mathbf{z} = f_\theta(\mathbf{x})$, với f_θ là một CNN pretrained đã bị đóng băng, và dự đoán lớp 1 khi $p \geq 0.5$. Ranh giới quyết định có dạng gì?

A Siêu phẳng trong không gian pixel, vì logistic regression là tuyến tính

B Siêu phẳng theo embedding, còn theo pixel thì có thể cong tùy ý

C Không là siêu phẳng ở đâu cả, vì sigmoid là hàm phi tuyến

D Siêu phẳng theo cả embedding lẫn pixel, vì encoder đã bị đóng băng

PHỎNG THEO CMU 10-601 · Exam 2 Practice · Q2.3(b)

LUYỆN ĐỀ 2

Kho tìm kiếm có 50 000 vector, mỗi vector 100 số `float32`. Lưu sẵn ma trận cosine `S` cho mọi cặp thì tốn bao nhiêu bộ nhớ, so với chính ma trận embedding `W`?

A `S` khoảng 20 MB, bằng `W`

B `S` khoảng 20 GB, gấp 1000 lần `W`

C `S` khoảng 10 GB, gấp 500 lần `W`

D `S` khoảng 2.5 GB, gấp 125 lần `W`

PHỎNG THEO USA-NA-AIO 2026 · Vòng 1 · Part 5.14

LUYỆN ĐỀ 3

Matrix factorization trong hệ gợi ý học vector cho người dùng và vector cho phim, rồi so chúng trực tiếp bằng tích vô hướng. Một đội muốn làm tương tự: lấy embedding ảnh từ một ResNet và embedding mô tả từ DistilBERT, dùng PCA đưa cả hai về 128 chiều, rồi tìm ảnh bằng cosine với mô tả. Câu nào đúng?

A Làm được, vì sau PCA hai loại vector cùng có 128 chiều

B Làm được, nếu chuẩn hóa L2 cả hai loại vector trước

C Không được, vì tìm láng giềng chỉ dùng Euclid, không dùng cosine

D Không được, vì hai encoder không học chung một không gian

PHỎNG THEO CMU 10-601 · Exam 3 Practice · Q7.4

Chốt ý

Feature extraction giữ encoder đứng yên, học head nhỏ hoặc dùng kNN trên embedding.

Cosine so hướng; pooling phải bỏ padding; fit mọi bước trên train và chia theo nhóm khi ảnh gần trùng.

Sau giờ học

Làm **10 câu kiểm tra cuối bài** trên trang bài học.

Chạy notebook: đổi `n_components` thành 8 và 64, đoán trước kNN đổi thế nào rồi kiểm lại.